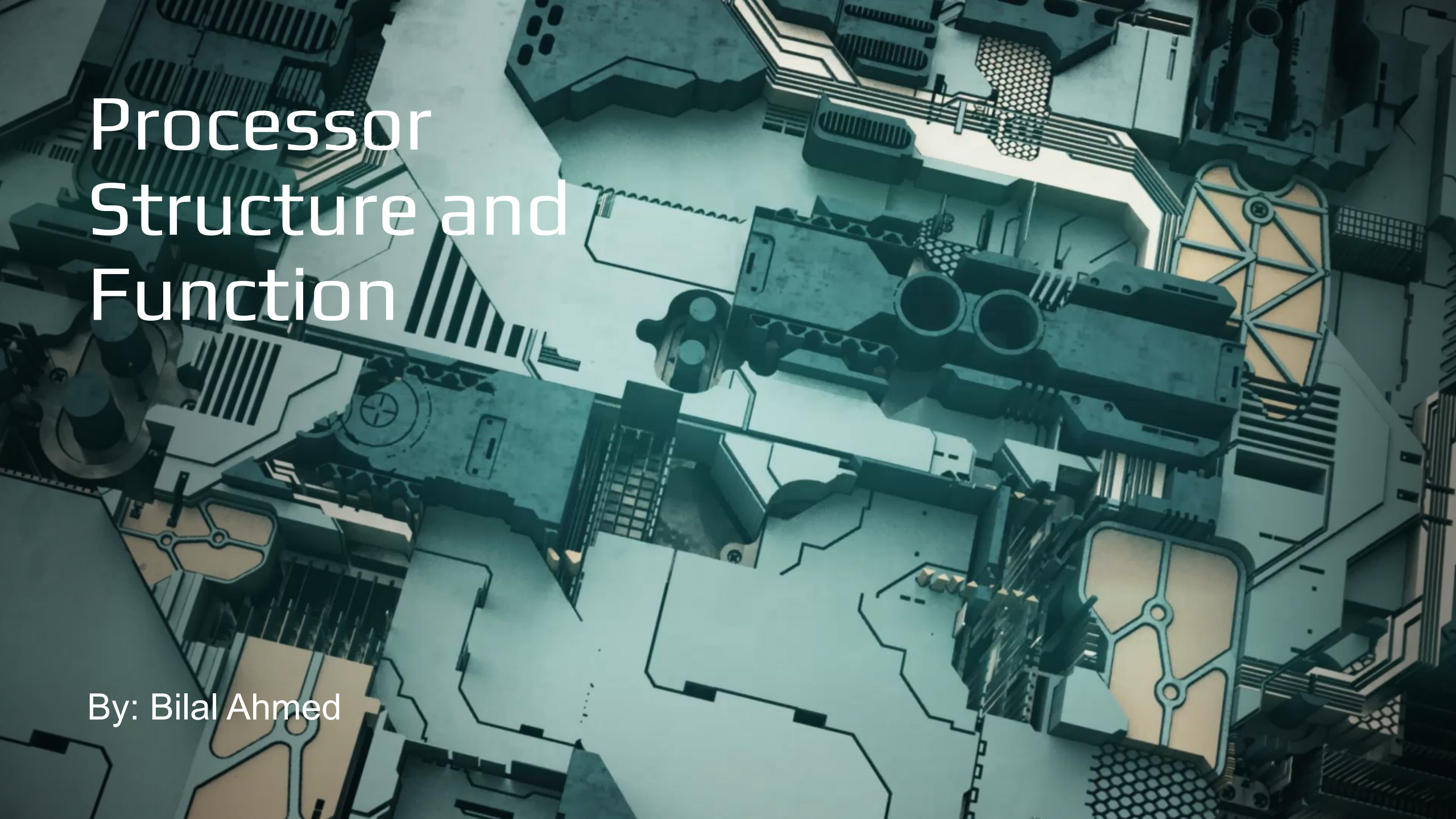




Processor Structure and Function

By: Bilal Ahmed

Processor Requirements

1

Fetch Instruction:
Processor reads the next instruction from memory.

2

Decode Instruction:
Processor interprets the instruction to determine the required action.

3

Fetch Operands: If needed, processor retrieves data from memory or an I/O device.

4

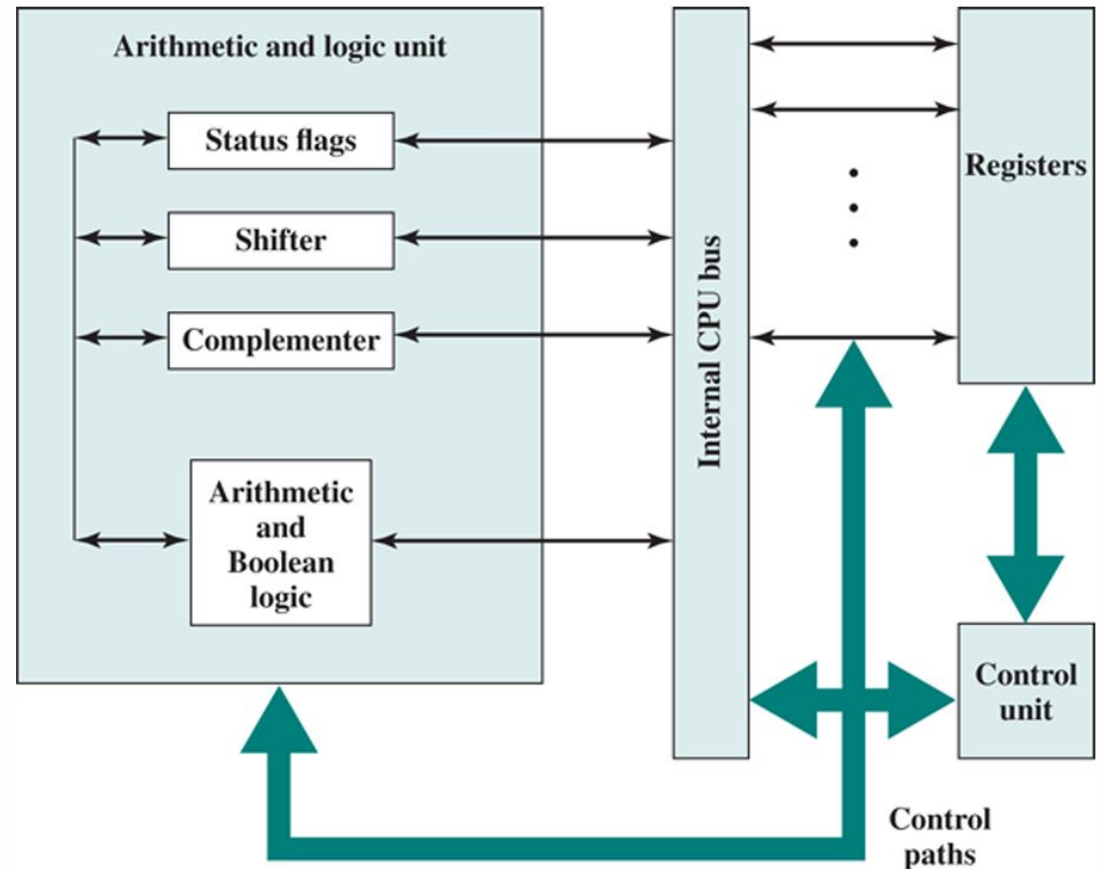
Execute Operation:
Processor performs the required arithmetic, logic, or control action.

5

Write Back:
Processor stores the result in memory or sends it to an I/O device.

Internal Structure of the CPU

- **ALU:** Performs all arithmetic and logical operations in the processor.
- **Control Unit (CU):** Directs data movement and controls how instructions are executed.
- **Registers:** Small, fast storage locations used to hold data and instructions temporarily.
- **Internal Processor Bus:** Pathway that transfers data among registers, ALU, and control unit.
- **Processor Structure:** Mirrors the overall computer structure—core units connected by data paths.



Register Organization

Memory Hierarchy Reminder: Higher levels are faster, smaller, and more expensive.

Processor Registers: Act as the fastest memory level above cache and main memory.

User-Visible Registers: Used directly by programmers to reduce memory access and improve performance.

Control & Status Registers: Used by the control unit and operating system to manage instruction execution.

Note: Separation isn't strict—some registers (like the program counter) may be user-visible on some architectures.

User Visible Registers



General-Purpose Registers (GPRs): Flexible registers used for a wide range of operations (arithmetic, logic, data storage). Programmers choose how to use them.



Data Registers: Specifically used to hold numeric values, characters, or intermediate results during computation.



Address Registers: Hold memory addresses used for accessing data or instructions (e.g., pointers, index registers, stack pointer).

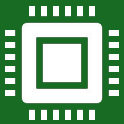


Condition Code Registers (Flags): Hold status information about the result of operations (e.g., zero, carry, overflow), used for decision-making and branching.

Condition Codes



Condition Code Bits: Status flags (e.g., zero, carry, overflow) stored in special registers; instructions can read them but programmers usually cannot modify them directly.



Access Behavior: These bits are updated automatically by the processor after operations and are referenced implicitly by conditional instructions.



Architectures Without Condition Codes: Some modern processors (e.g., IA-64, MIPS) avoid condition code registers entirely; instead, each conditional instruction performs its own comparison and decides the branch outcome directly, without storing flags.

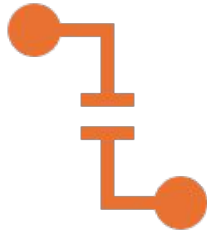
Condition Codes

Feature	With Condition Codes	Without Condition Codes
Where status is stored	In a flags register (Zero, Carry, Overflow, Sign)	No flags register; status not stored globally
How branching works	Branch instructions read flags set by previous operations	Each branch instruction performs its own comparison
Instruction dependency	High — operations affect flags used by later instructions	Low — no hidden dependencies on previous instructions
Programming style	Compact but harder to analyze; hidden side effects	More explicit, easier to optimize and pipeline
Examples	x86, ARM (older versions), 8086	MIPS, IA-64, RISC-V (uses compare and branch)

Example

Feature	x86 / EMU8086 (CISC)	RISC-V (RISC)
How comparisons work	Separate instruction (CMP) sets FLAGS	Comparison done inside the branch instruction
Uses condition codes?	Yes (ZF, CF, SF, OF)	No flags
Dependency	Branch depends on FLAGS set earlier	Branch depends only on register values
Example	CMP AX,BX then JE	Single beq x10,x11,label
Pipeline behavior	Hidden dependencies → harder to pipeline	Explicit operands → easier to pipeline

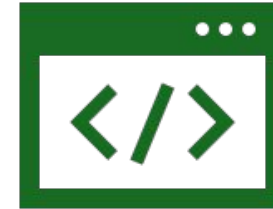
Comparision



With Condition Codes

Advantage: Allows compact instructions by storing status centrally.

Disadvantage: Creates hidden dependencies that slow optimization and pipelining.



Without Condition Codes

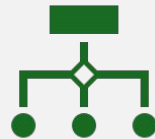
Advantage: Makes branching explicit, improving clarity and pipeline efficiency.

Disadvantage: Requires more comparison operations, increasing instruction count.

Control and Status Registers



Processors contain special registers used to **manage and control instruction execution**; most are **not user-visible**.



Different architectures organize these registers differently, but most share a similar set of essential control registers.

Key Registers Required for Instruction Execution

Program Counter (PC): Holds the address of the **next instruction** to fetch.

Instruction Register (IR): Holds the **current instruction** being decoded/executed.

Memory Address Register (MAR): Holds the **address in memory** to read from or write to.

Memory Buffer Register (MBR): Holds the **data being transferred** to or from memory.

Buffering Hardware

Not all CPUs use the names *MAR* and *MBR*

But every processor includes **equivalent buffering hardware** to temporarily hold data and addresses before bus transfers.

Common Flags in the Program Status Word (PSW)



Sign (S): Stores the sign bit of the last arithmetic result.



Zero (Z): Set when the result of an operation is 0.



Carry (C): Set if an addition generates a carry or a subtraction generates a borrow; useful in multiword arithmetic.



Equal (E): Set if a logical comparison shows equality.



Overflow (O): Indicates arithmetic overflow.

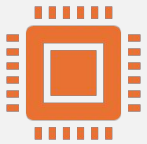


Interrupt Enable/Disable (I): Controls whether interrupts are enabled or disabled.



Supervisor (S/V): Indicates whether the processor is in supervisor (privileged) or user mode; affects execution of certain instructions and memory access.

Instruction Cycle



Fetch: Read the next instruction from memory into the processor.



Execute: Interpret the opcode and perform the indicated operation.

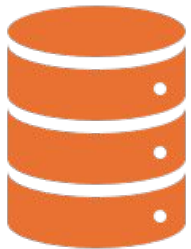


Interrupt: If interrupts are enabled and an interrupt has occurred, save the current process state and service the interrupt.



Instruction Cycle Expansion: An additional stage, the **Indirect Cycle**, is included in the instruction cycle.

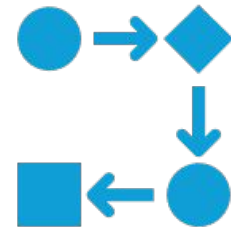
Indirect Cycle Purpose



Some instructions require operands stored in memory.



Indirect addressing may require *extra memory accesses* to obtain the actual operand address.



Each indirect access acts like an additional mini-stage in the cycle.

Simple Analogy

Think of it like someone telling you:

“Your gift is in the box at **Locker 500.**”

You open Locker 500 and find a note:

“Actually, the gift is in **Locker 1200.**”

So you go to Locker 1200 to get the real item.

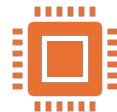
Flow of the Instruction Cycle



Alternates between **Instruction Fetch** and **Instruction Execution**.



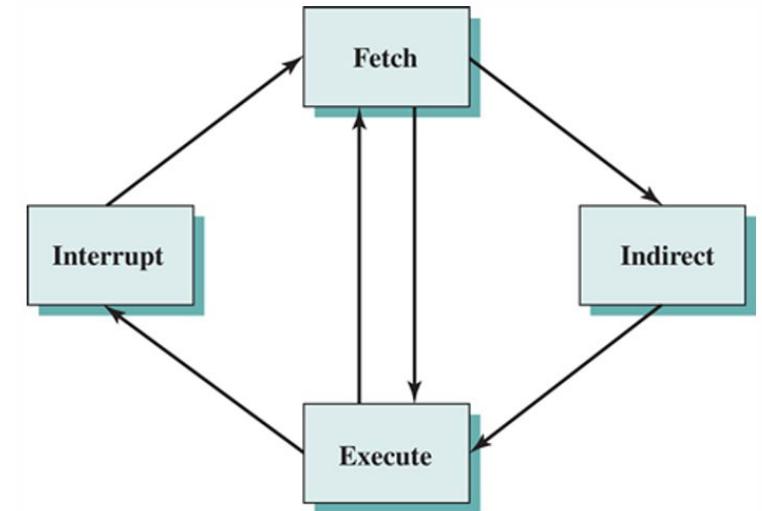
After fetching an instruction, the processor checks whether **indirect addressing** is used.



If indirect addressing is present, the processor performs **one or more operand fetches** through indirect addressing.

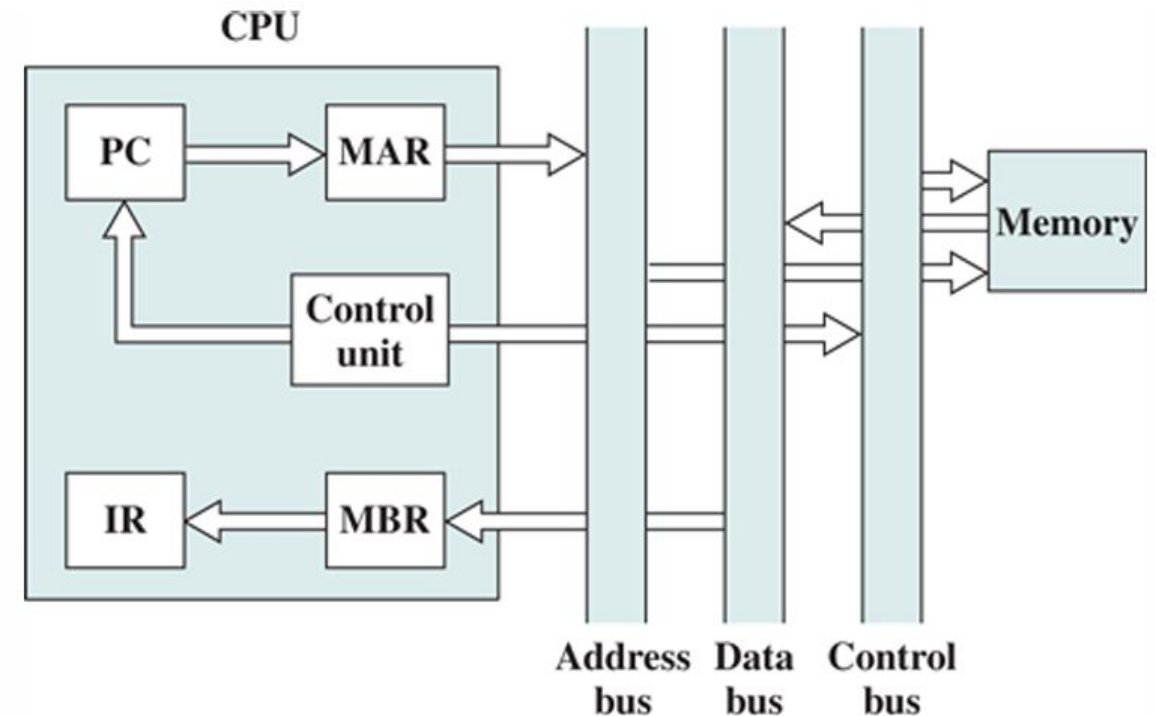


After instruction execution, **interrupts may be processed** before the next instruction fetch begins.



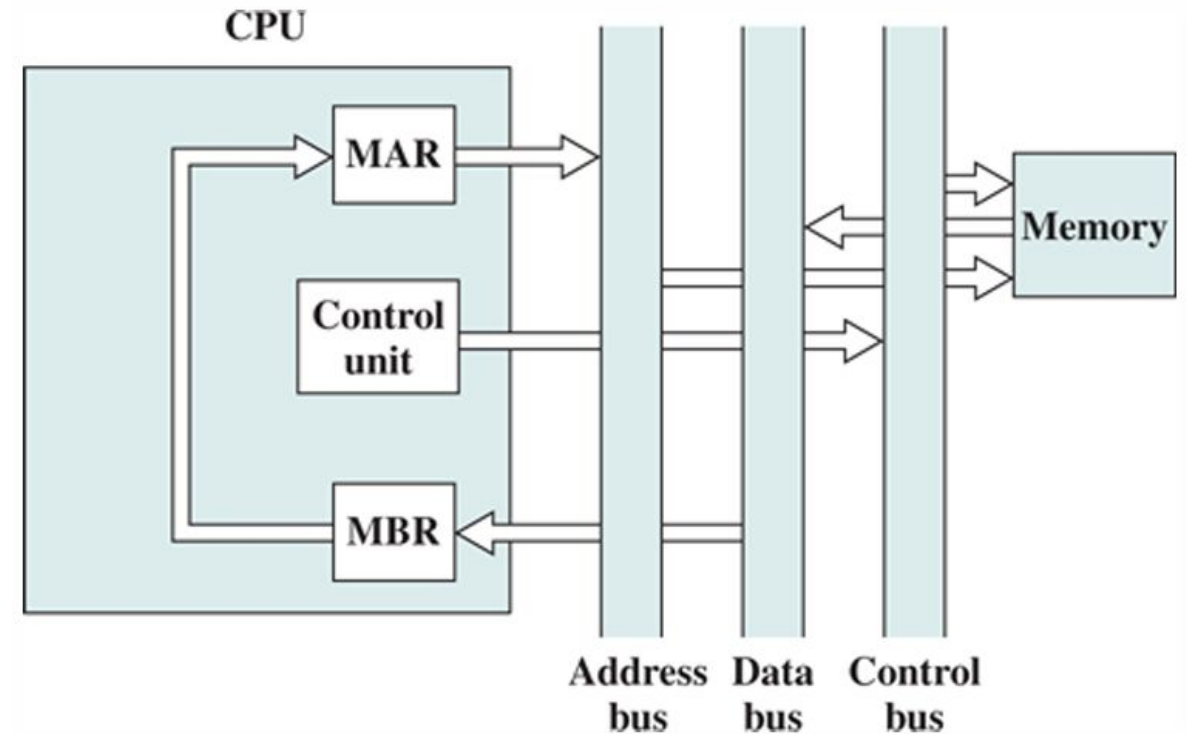
Data Flow during the Fetch Cycle

- **PC → MAR:** The PC sends the next instruction address to the MAR, which places it on the address bus.
- **Memory Read:** The control unit issues a read signal, and memory outputs the instruction onto the data bus.
- **MBR → IR:** The instruction from the data bus is loaded into the MBR and then moved to the IR.
- **PC++:** The PC is incremented to point to the next instruction.



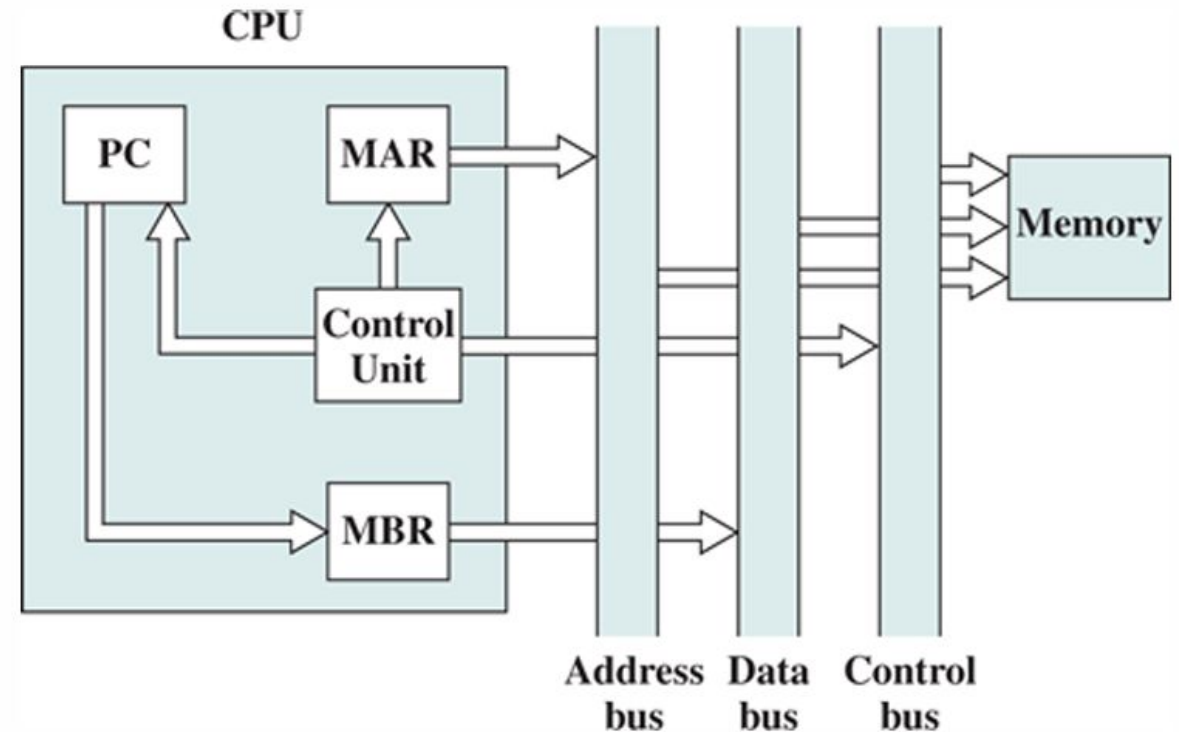
Data Flow during the Indirect Cycle

- Check IR:** After the fetch cycle, the control unit checks the IR to see if indirect addressing is used.
- MBR → MAR:** If indirect addressing is required, the address field in the MBR (right-most N bits) is copied to the MAR.
- Memory Read:** The control unit issues a memory read to access the actual operand address.
- Load to MBR:** The fetched operand address is loaded into the MBR, completing the indirect cycle.



Data Flow during the Interrupt Cycle

- Save PC:** The current contents of the PC are copied to the MBR so they can be written to memory.
- Select Save Location:** The control unit loads the MAR with the special memory location reserved for saving the PC (often a stack pointer).
- Store PC:** The value in the MBR is written to memory.
- Load Interrupt Handler:** The PC is loaded with the starting address of the interrupt service routine, so the next cycle fetches the interrupt-handling instruction.



Data Flow during the Execution Cycle

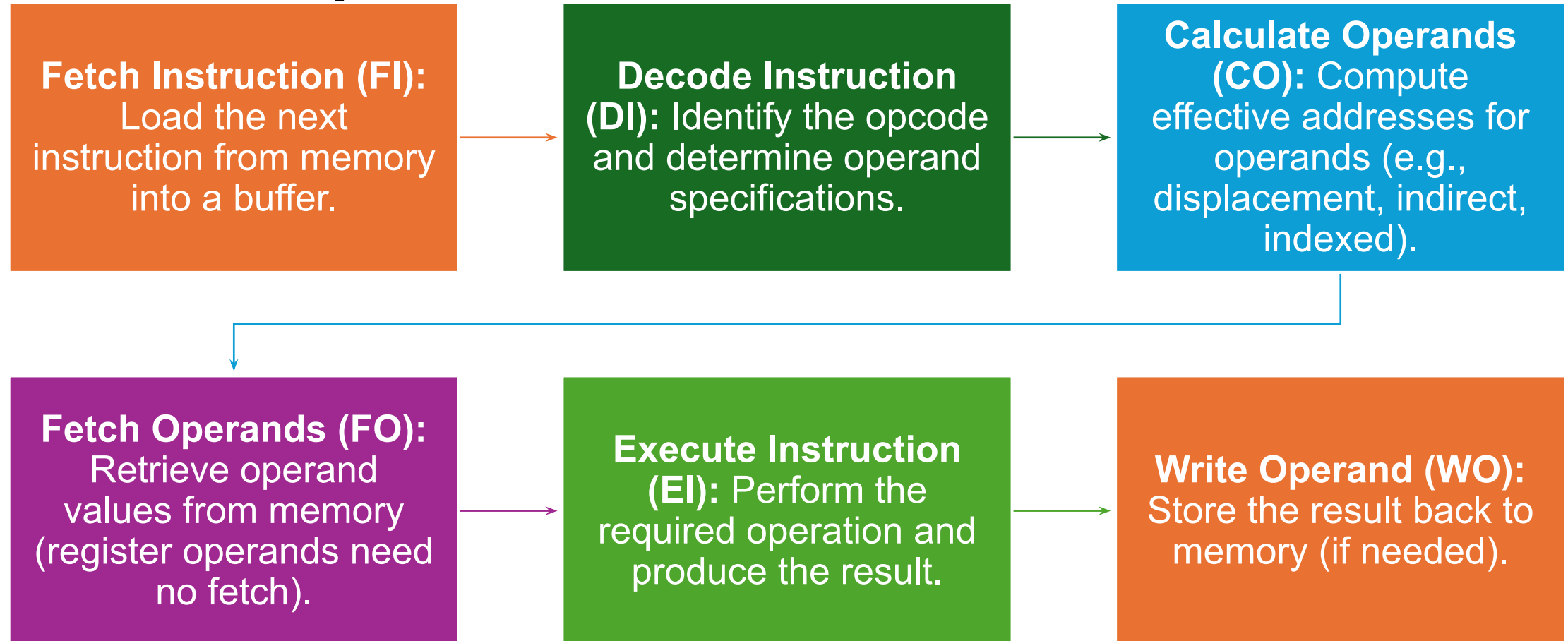
Execute cycle varies: Unlike fetch and indirect cycles, execution depends on the specific instruction in the IR.

Register operations: May involve transferring data between internal registers.

Memory or I/O access: May require reading from or writing to memory or I/O devices.

ALU operations: May invoke the ALU for arithmetic or logic processing.

Instruction Processing Decomposition



Instruction Pipelining



Processor performance improves not only through faster hardware but also through better internal organization.



One key organizational enhancement is **instruction pipelining**.



Pipelining overlaps different stages of multiple instructions (fetch, decode, execute, etc.).



This overlap increases **instruction throughput**, allowing more instructions to finish in less time.

Pipelining Strategy



Instruction pipelining works like an **assembly line**, where each stage processes part of the work.



While one instruction is being fetched, another can be decoded, and another executed.



Multiple instructions are therefore **in progress simultaneously**, each at a different stage.



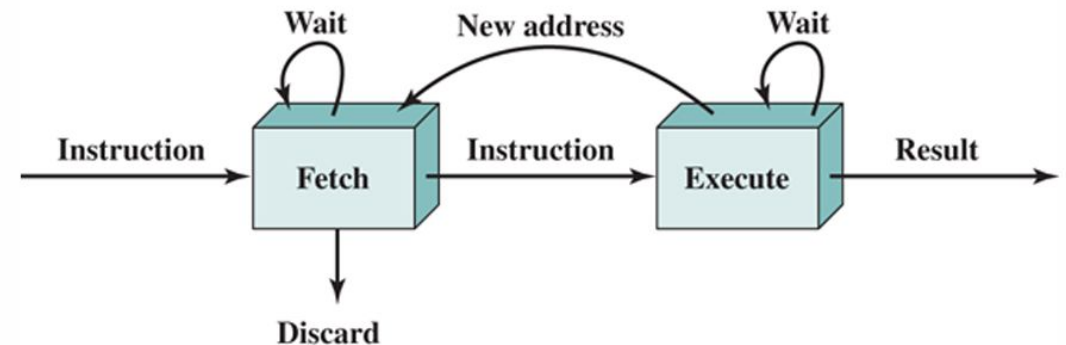
Like a real pipeline, new instructions enter before earlier ones finish, increasing overall throughput.

Instruction Prefetch & Basic Pipelining

- An instruction actually goes through multiple steps, so parts of its execution can overlap.
- A simple pipeline divides processing into **two stages**: *instruction fetch* and *instruction execute*.
- While one instruction is being executed, the processor can use idle memory cycles to **fetch the next instruction early**.
- This technique is called **instruction prefetch** or **fetch overlap**, and it works by buffering the fetched instruction.
- Because of this buffering, the processor requires **additional registers** to hold prefetched instructions.



(a) Simplified view



(b) Expanded view

Pipeline Latches



Pipelining uses fast intermediate registers called **latches** to store values between stages.



These latches **decouple pipeline stages**, allowing each stage to work independently.



A **common clock** triggers all latches simultaneously, moving data from one stage to the next every cycle.

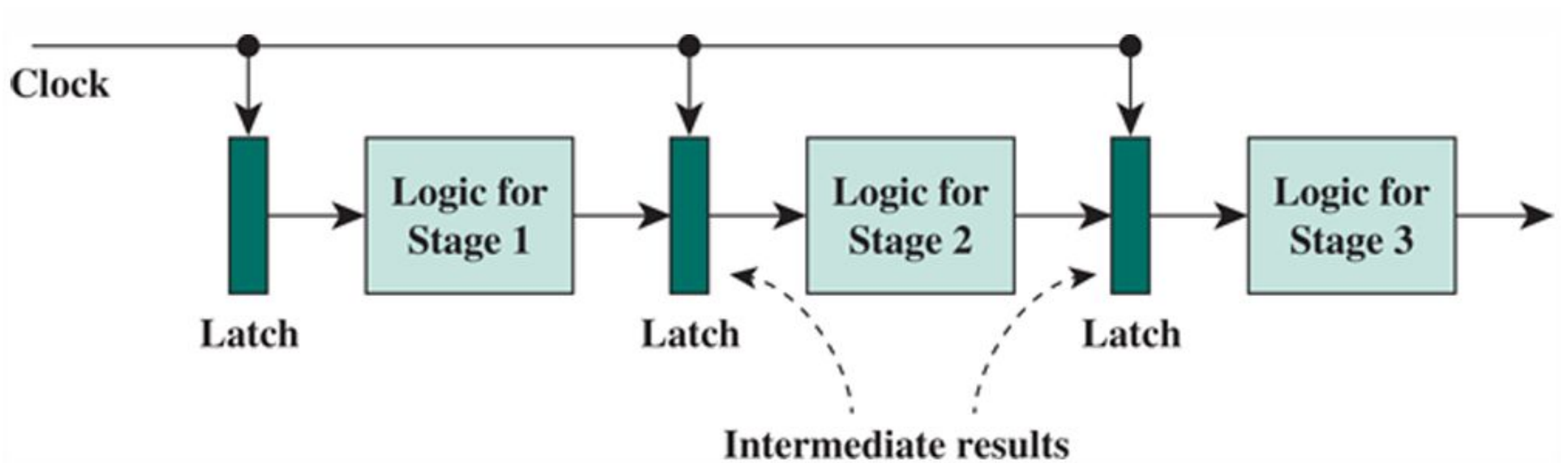


At each clock edge, latches capture their inputs and pass them forward for the next stage to process.



This simplified model hides additional practical complexities (e.g., hazards, forwarding).

Pipeline Latches



Pipeline Performance with Six Stages

- By breaking instruction execution into **six smaller, balanced stages**, each stage takes roughly the same time.
- With all stages operating in parallel (pipelined), multiple instructions are processed at once.
- Under the assumption of equal stage duration, a six-stage pipeline can reduce the time to execute **9 instructions** from **54 time units (no pipeline)** to just **14 time units (pipelined)**.
- This demonstrates how pipelining greatly improves **instruction throughput**, even though individual instructions still require multiple stages.

Time →

	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Instruction 1	FI	DI	CO	FO	EI	WO								
Instruction 2		FI	DI	CO	FO	EI	WO							
Instruction 3			FI	DI	CO	FO	EI	WO						
Instruction 4				FI	DI	CO	FO	EI	WO					
Instruction 5					FI	DI	CO	FO	EI	WO				
Instruction 6						FI	DI	CO	FO	EI	WO			
Instruction 7							FI	DI	CO	FO	EI	WO		
Instruction 8								FI	DI	CO	FO	EI	WO	
Instruction 9									FI	DI	CO	FO	EI	WO

Limitations



Execution Stage Is Usually Slower

- Execution involves operand fetch, computation, and storing results.
- Since execution takes longer than instruction fetch, the **fetch stage often waits**, reducing pipeline efficiency.

Conditional Branches Break the Flow

- For branch instructions, the **next instruction address is unknown** until the branch is resolved.
- Fetch must wait for the execute stage to decide the branch outcome.
- Execute may then need to wait for the next instruction to be fetched.

Branch Guessing



To reduce delays caused by conditional branches, the pipeline can **guess** the next instruction.



A simple rule: when a branch enters the execute stage, the fetch stage **predicts “branch not taken”** and fetches the next sequential instruction.



If the branch is **not taken**, the guess is correct and no time is lost.



If the branch **is taken**, the fetched instruction is discarded and the correct target instruction must be fetched, causing a small delay.